4º ANO CIÊNCIA DA COMPUTAÇÃO

Faculdades Integradas de Bauru

Tópicos Especiais II

MONITORAMENTO DE INFRAESTRUTURA COM PROMETHEUS GRAFANA ZABBIX E PODMAN

Luan Alves Camargo Marques
Matheus Andrade Danno
Fabricio bastos dos santos

Bauru
2026

MONITORAMENTO DE INFRAESTRUTURA COM PROMETHEUS GRAFANA ZABBIX E PODMAN

Relatório técnico-científico apresentado à disciplina de Tópicos Especiais II do curso de Ciência da Computação das Faculdades Integradas de Bauru, como registro do planejamento, implementação e avaliação de um laboratório de monitoramento de infraestrutura.

Bauru
2026

RESUMO

Este trabalho apresenta o planejamento, a implementação e a avaliação de um laboratório controlado de monitoramento de infraestrutura executado em uma estação Fedora. A solução utiliza Podman Compose para orquestrar a aplicação OWASP Juice Shop, clientes automatizados de tráfego, Node Exporter, Prometheus, Grafana, Zabbix e um dashboard próprio em Flask. O objetivo foi observar CPU, memória, disco, rede e disponibilidade, comparando o modelo de coleta por scraping do Prometheus com a visualização e o gerenciamento de eventos do Zabbix. O Node Exporter foi configurado para observar o host Fedora e não apenas os containers. O Prometheus coletou as métricas a cada 15 segundos, avaliou uma regra de CPU alta e forneceu dados ao Grafana e ao dashboard centralizado. O Zabbix foi instalado em containers e preparado para consultar as métricas por HTTP por meio da API do Prometheus, alternativa adotada devido à indisponibilidade direta do pacote zabbix-agent2 nos repositórios usados no Fedora. As evidências demonstram a execução simultânea dos serviços, a disponibilidade dos targets monitorados e a consolidação das fontes de dados em uma interface própria. Conclui-se que a composição proposta é reproduzível, de baixo acoplamento com o host e adequada para fins didáticos de DevOps e monitoramento de infraestrutura.

Palavras-chave: monitoramento de infraestrutura; Prometheus; Grafana; Zabbix; Podman; DevOps.

SUMÁRIO

1 INTRODUÇÃO

2 FUNDAMENTAÇÃO TÉCNICA

3 METODOLOGIA E ARQUITETURA DO LABORATÓRIO

4 IMPLEMENTAÇÃO DA SOLUÇÃO

5 RESULTADOS E DISCUSSÃO

6 LIMITAÇÕES E TRABALHOS FUTUROS

7 CONCLUSÃO

REFERÊNCIAS

APÊNDICE A – COMANDOS PRINCIPAIS

1 INTRODUÇÃO

O monitoramento de infraestrutura é uma prática indispensável para observar o estado operacional de servidores, aplicações e redes. Sem métricas e eventos confiáveis, problemas de capacidade, indisponibilidade ou degradação de desempenho tendem a ser identificados tardiamente. Em ambientes de DevOps, a instrumentação e a visualização contínua reduzem a dependência de verificações manuais e apoiam decisões fundamentadas durante incidentes e mudanças de configuração.

A proposta inicial da disciplina previa a instalação de serviços em uma máquina virtual Ubuntu Server. Após alinhamento com o professor, o escopo foi adaptado para uma estação Fedora, sem máquina virtual, mantendo os objetivos de coleta de CPU, memória RAM, disco, rede e disponibilidade. A decisão preservou a natureza controlada do laboratório e permitiu concentrar o estudo na orquestração de containers, na observabilidade e na comparação entre ferramentas livres.

A questão orientadora foi: como integrar Prometheus, Grafana e Zabbix para observar o mesmo ambiente local, gerar tráfego controlado para uma aplicação web e apresentar os resultados de forma compreensível? Para respondê-la, foi construído um ambiente reprodutível com Podman Compose, uma aplicação OWASP Juice Shop, containers clientes, um bot de tráfego e uma camada adicional de consolidação em Flask.

1.1 Objetivos

- implementar um laboratório reproduzível de monitoramento no Fedora usando containers;
- coletar métricas de CPU, memória, disco, rede e disponibilidade do host;
- gerar tráfego HTTP contínuo contra a aplicação monitorada;
- comparar os papéis de Prometheus, Grafana e Zabbix;
- centralizar indicadores e identificar, em cada visualização, a fonte do dado exibido.

2 FUNDAMENTAÇÃO TÉCNICA

2.1 Monitoramento e séries temporais

Uma métrica representa uma observação numérica associada a um instante de tempo. Em monitoramento de infraestrutura, exemplos frequentes são porcentagem de CPU ocupada, memória disponível, bytes transmitidos por interface e estado de disponibilidade. O armazenamento em séries temporais permite correlacionar alterações de comportamento com eventos operacionais, como o início de uma carga de requisições ou a parada de um serviço.

2.2 Prometheus e Node Exporter

O Prometheus é um sistema de monitoramento e alerta que coleta dados por scraping de endpoints HTTP e os armazena como séries temporais identificadas por rótulos. Sua arquitetura se adapta bem a métricas numéricas e ambientes orientados a serviços (PROMETHEUS AUTHORS, 2026). No laboratório, o Node Exporter expõe métricas do Fedora em formato Prometheus; o servidor Prometheus as coleta periodicamente e disponibiliza consultas em PromQL.

2.3 Zabbix

O Zabbix é uma solução de monitoramento open source que acompanha parâmetros de servidores, serviços, aplicações e redes, oferecendo histórico, problemas, triggers e frontend web (ZABBIX SIA, 2026). Neste projeto, o Zabbix foi executado com PostgreSQL e frontend próprios. Para evitar instalar dependências adicionais no Fedora, os itens do Zabbix foram projetados como consultas HTTP à API do Prometheus, mantendo o mesmo host como referência de comparação.

2.4 Grafana e dashboards

Grafana organiza fontes de dados em painéis e dashboards, permitindo apresentar gráficos, indicadores e tabelas em uma única interface (GRAFANA LABS, 2026). O seu uso no laboratório é complementar ao Prometheus: o primeiro fornece visualização, enquanto o segundo fornece armazenamento e consulta de métricas. Um dashboard Flask adicional foi criado para explicitar a origem de cada dado e combinar status de Prometheus, Zabbix e Juice Shop.

2.5 Containers e Podman

Podman é uma ferramenta para criar e manter containers, podendo ser utilizada sem daemon central e em modo não privilegiado. A documentação recomenda a referência

explícita das imagens, prática adotada neste trabalho para tornar a origem dos componentes clara (PODMAN, 2026). O arquivo compose.yaml descreve os serviços, a rede interna e os volumes persistentes do laboratório.

3 METODOLOGIA E ARQUITETURA DO LABORATÓRIO

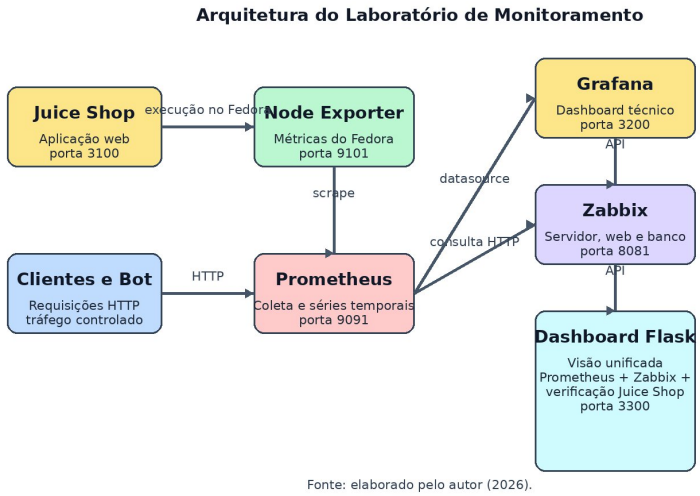
A pesquisa caracteriza-se como aplicada e experimental, pois constrói uma solução funcional e a observa em ambiente controlado. O cenário foi limitado à máquina local para reduzir variáveis de infraestrutura externa. A validação foi realizada por meio de verificações HTTP, estado dos containers, targets do Prometheus e inspeção das interfaces web.

3.1 Ambiente de execução

Componente	Versão ou característica	Finalidade
Sistema operacional	Fedora 44, kernel 7.2.5	Host monitorado e plataforma de execução
Podman	5.8.4	Execução de containers
podman-compose	1.6.0	Leitura do arquivo Compose
Prometheus	Container oficial	Scraping, consultas e regra de alerta
Node Exporter	Container oficial	Exposição de métricas do Fedora
Grafana	Container oficial	Dashboard técnico
Zabbix	Server, web e PostgreSQL	Eventos, itens e frontend
OWASP Juice Shop	Imagem oficial	Aplicação web monitorada

3.2 Arquitetura

A Figura 1 apresenta o fluxo principal. Clientes e bot enviam requisições à aplicação. O Node Exporter acessa informações do host por montagens somente leitura de /proc, /sys e /, e o Prometheus coleta o endpoint exposto. Grafana e dashboard Flask consultam o Prometheus. O Zabbix utiliza consultas HTTP para obter os valores necessários e apresenta itens e eventos em seu frontend.



Fonte: elaborado pelo autor (2026).

Figura 1 – Arquitetura do laboratório

Fonte: elaborado pelo autor (2026)

4 IMPLEMENTAÇÃO DA SOLUÇÃO

4.1 Orquestração dos serviços

Todos os componentes foram descritos no compose.yaml e conectados à rede interna denominada monitoramento. As portas do host foram escolhidas para evitar colisão com serviços já existentes na máquina: 3100 para Juice Shop, 3200 para Grafana, 3300 para o dashboard Flask, 8081 para Zabbix Web, 9091 para Prometheus e 9101 para Node Exporter. O PostgreSQL do Zabbix permaneceu restrito à rede interna.

Serviço	Porta local	Função
Juice Shop	3100	Aplicação web monitorada
Grafana	3200	Dashboard técnico
Dashboard Flask	3300	Visão centralizada com fontes identificadas
Zabbix Web	8081	Frontend de monitoramento e eventos
Prometheus	9091	Consultas, targets e alertas
Node Exporter	9101	Métricas brutas do Fedora

4.2 Coleta de métricas

O Prometheus foi configurado com intervalo de coleta e avaliação de 15 segundos. Dois jobs foram definidos: o próprio Prometheus e o host Fedora, identificado como fedora-node. A regra CPUAlta calcula a média de tempo não ocioso do processador usando node_cpu_seconds_total e gera condição de alerta acima de 80% durante dois minutos.

4.3 Geração de tráfego

Dois containers Alpine executam buscas periódicas por apple e banana na API do Juice Shop. Um terceiro serviço, bot-trafego, simula navegação básica, requisitando a página inicial e termos de busca distintos aproximadamente uma vez por segundo. Essa estratégia produz tráfego de rede e atividade de aplicação sem instalar ferramentas de carga no host Fedora.

4.4 Dashboard centralizado

O serviço Flask consulta a API do Prometheus para indicadores instantâneos, séries de CPU e memória, além dos targets ativos. Também realiza uma verificação HTTP do Juice Shop. A consulta da API do Zabbix é opcional e depende de credencial local no arquivo .env, que não é versionado. Cada cartão, gráfico ou tabela informa explicitamente sua fonte, evitando que o usuário confunda métricas brutas, agregações PromQL e eventos do Zabbix.

5 RESULTADOS E DISCUSSÃO

5.1 Visão centralizada

A Figura 2 registra o dashboard Flask em execução. Na captura, a disponibilidade do Node Exporter aparece como UP e a verificação HTTP do Juice Shop também aparece como UP. O painel mostra indicadores de CPU, memória, disco e rede, gráficos de CPU e memória e a tabela de targets do Prometheus. A captura registra CPU de 27,5%, memória de 52,6% e disco de 20,5%; esses valores representam um instante do laboratório e não devem ser interpretados como média de longo prazo.

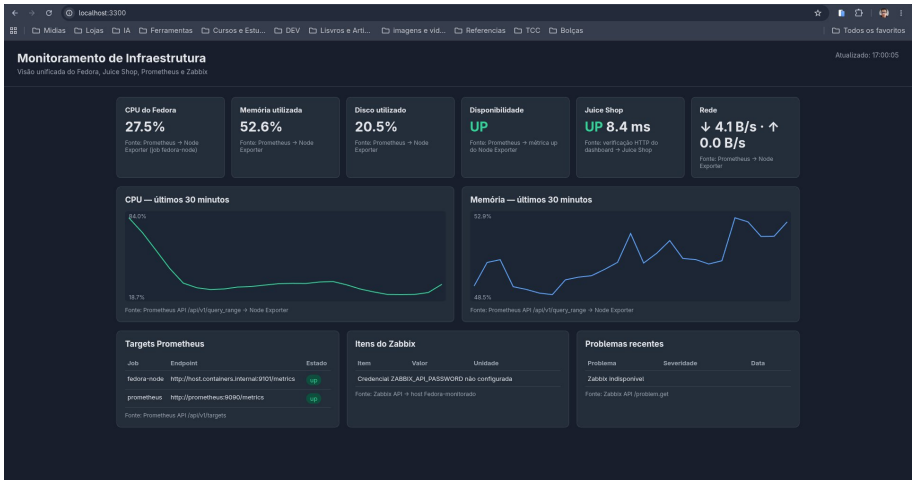


Figura 2 – Dashboard centralizado em Flask

Fonte: captura do ambiente local fornecida pelo autor (2026)

5.2 Targets do Prometheus e disponibilidade

A tabela de targets do dashboard confirma que os jobs prometheus e fedora-node estavam ativos. O endpoint do host foi resolvido internamente como host.containers.internal:9101/metrics. Esse resultado demonstra que o Prometheus conseguiu alcançar o exporter exposto pelo host e que a coleta não ficou limitada ao container da aplicação.

A regra CPUAlta foi mantida como evidência de alerta baseado em série temporal. Sua lógica exige permanência da condição acima do limiar por dois minutos, reduzindo o risco de alertas provocados por picos transitórios. O comportamento é adequado para a demonstração prática prevista no laboratório, na qual uma carga controlada pode ser aplicada e posteriormente interrompida.

5.3 Métricas expostas pelo Node Exporter

A Figura 3 reúne as evidências de coleta e visualização. O painel superior esquerdo mostra os targets do Prometheus, ambos com valor 1; o painel superior direito apresenta parte

do endpoint /metrics do Node Exporter, em formato textual com linhas HELP e TYPE; e o painel inferior mostra o Grafana. Nesse dashboard, a série de CPU registra um pico próximo de 95%, produzido pelo experimento controlado de carga, enquanto memória, disco, rede e a quantidade de targets permanecem visíveis.

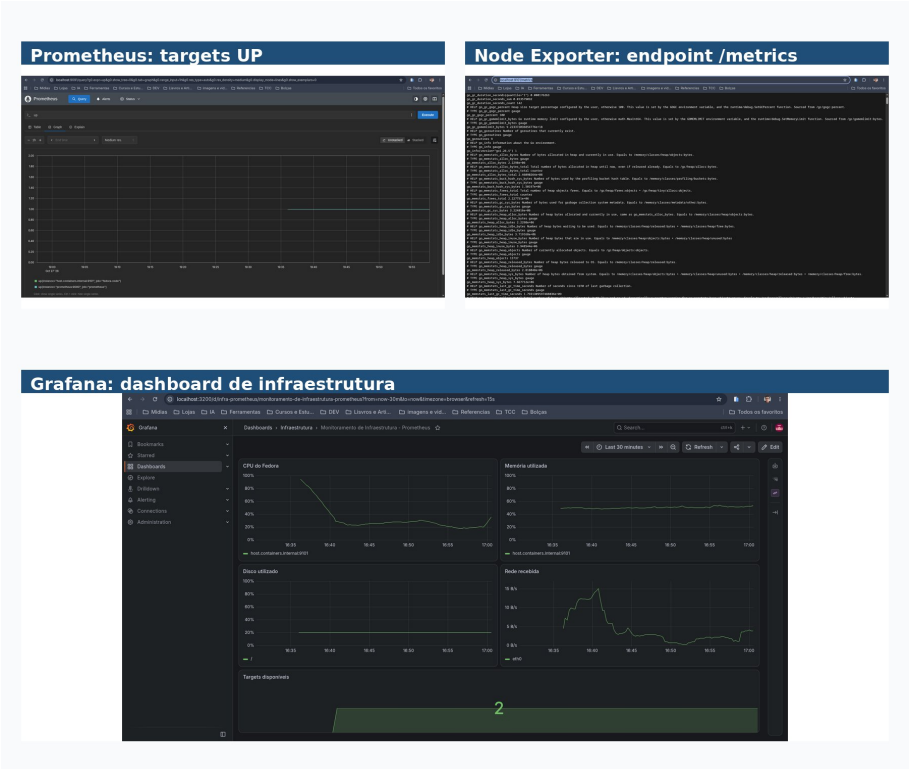


Figura 3 – Evidências de coleta e visualização com Prometheus, Node Exporter e Grafana
Fonte: capturas do ambiente local fornecidas pelo autor (2026)

5.4 Aplicação e tráfego

A aplicação OWASP Juice Shop foi escolhida como alvo por ser uma aplicação web pronta para execução em container e apropriada para laboratórios. A OWASP descreve o projeto como aplicação deliberadamente insegura voltada a treinamento; por esse motivo, ela foi mantida restrita ao ambiente local, sem exposição pública (OWASP FOUNDATION, 2026). A verificação HTTP usada pelo dashboard mede a disponibilidade da aplicação, enquanto os containers clientes e o bot produzem requisições internas ao Compose.

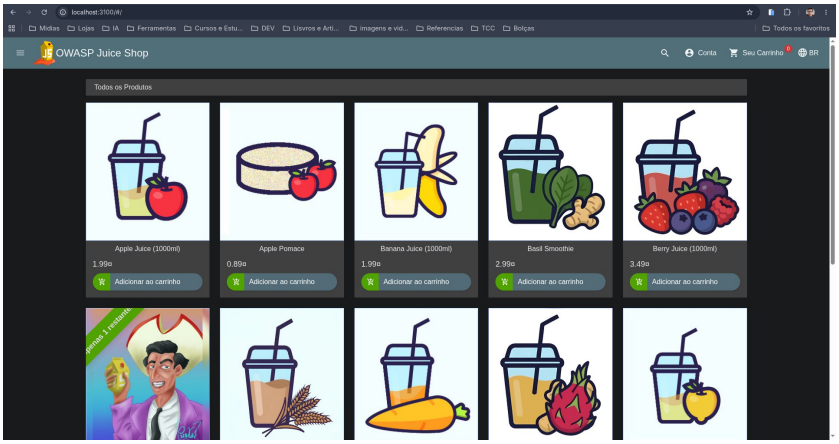


Figura 4 – Interface da aplicação OWASP Juice Shop

Fonte: captura do ambiente local fornecida pelo autor (2026)

5.5 Zabbix e comparação entre as ferramentas

A Figura 5 confirma o cadastro do host Fedora-monitorado no frontend do Zabbix, com dois itens associados e estado habilitado. A solução foi instalada com servidor, interface web e banco PostgreSQL em containers. Os itens podem ser criados pelo script configurar_zabbix.py, que utiliza a API do Zabbix e não armazena senhas no repositório. A integração do dashboard Flask com a API do Zabbix depende da definição de ZABBIX_API_PASSWORD no arquivo .env local.

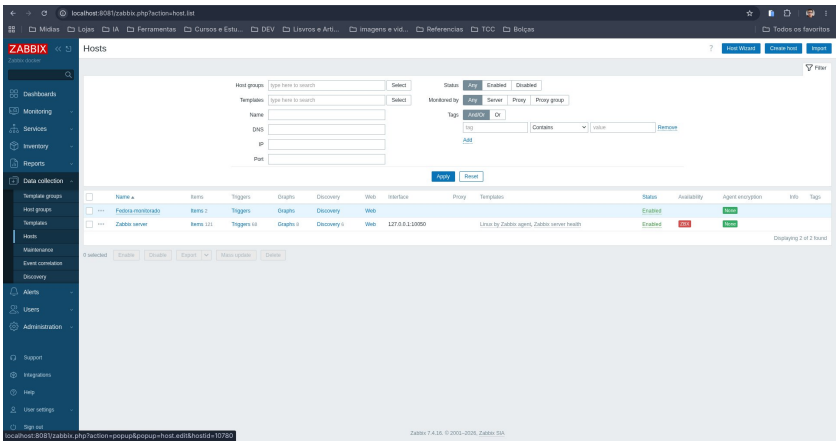


Figura 5 – Host Fedora-monitorado cadastrado no Zabbix

Fonte: captura do ambiente local fornecida pelo autor (2026)

Aspecto	Prometheus	Zabbix
Modelo principal	Scraping periódico de endpoints HTTP	Itens, histórico, triggers e frontend
Consulta	PromQL e API HTTP	Interface web e API JSON-RPC
Uso no laboratório	Coleta primária do Node Exporter	Eventos e consulta HTTP ao Prometheus
Visualização	Grafana e dashboard Flask	Frontend próprio e dashboard Flask
Alerta de CPU	Regra CPUAlta com condição temporal	Trigger baseada no item fedora.cpu.usage

A comparação deve considerar que os dados do Zabbix foram projetados para consulta ao Prometheus neste laboratório. Assim, as ferramentas não operam com coletores

completamente independentes: o Prometheus é a fonte primária da telemetria do Fedora e o Zabbix agrega a camada de itens, triggers e histórico a partir de consultas HTTP. Essa escolha foi tecnicamente adequada ao Fedora utilizado e mantém a comparação focada na organização e apresentação dos dados, não em uma disputa de precisão entre coletores distintos.

6 LIMITAÇÕES E TRABALHOS FUTUROS

A principal limitação é a dependência da credencial local para a leitura autenticada da API do Zabbix no dashboard Flask. A credencial foi propositalmente mantida fora do repositório, no arquivo .env ignorado pelo Git. Como consequência, uma captura do dashboard pode apresentar a mensagem de que a credencial não está configurada até que o usuário defina a senha localmente.

Outra limitação é que o teste de alta CPU registrado neste relatório utiliza o estado observado no momento da captura. Uma avaliação futura pode executar uma carga com duração controlada, registrar o instante de disparo dos alertas e exportar gráficos de ambas as ferramentas. A instrumentação nativa da aplicação, com métricas específicas de requisições HTTP, também permitiria quantificar o tráfego do bot de modo mais direto do que a observação de bytes de rede do host.

- configurar um serviço systemd de usuário para inicializar o Compose no login;
- finalizar a credencial do Zabbix no .env e validar os itens no dashboard Flask;
- registrar um experimento de carga de CPU com início, duração e recuperação;
- adicionar métricas específicas da aplicação ou um exporter de requisições HTTP;
- exportar o host, itens e triggers do Zabbix para anexação ao repositório.

7 CONCLUSÃO

O laboratório implementado demonstrou uma abordagem reproduzível de monitoramento de infraestrutura em Fedora sem necessidade de máquina virtual. O uso de Podman Compose reuniu a aplicação, os geradores de tráfego e os componentes de observabilidade em uma rede interna organizada, com portas locais controladas e volumes persistentes para dados relevantes.

O Node Exporter permitiu observar o host Fedora e o Prometheus confirmou a disponibilidade dos targets e a coleta de séries temporais. Grafana foi preparado como dashboard técnico, enquanto o dashboard Flask consolidou indicadores e tornou explícita a origem de cada informação. A disponibilidade do Juice Shop e o tráfego contínuo dos clientes

e do bot tornam o cenário adequado para demonstrações práticas de disponibilidade, CPU, memória, disco e rede.

A integração com Zabbix preserva seu papel de gerenciamento de itens e eventos, utilizando consultas HTTP como alternativa viável para o ambiente Fedora. Embora a configuração final da credencial da API permaneça uma etapa local, o desenho, os scripts e a documentação necessários foram entregues. Portanto, a solução atende ao objetivo didático de comparar tecnologias open source e demonstrar como dashboards facilitam a interpretação de métricas de infraestrutura.

REFERÊNCIAS

GRAFANA LABS. Grafana dashboards overview. Disponível em: https://grafana.com/docs/grafana/latest/fundamentals/dashboards-overview/. Acesso em: 27 out. 2026.

OWASP FOUNDATION. OWASP Juice Shop. Disponível em: owasp.org/projects/juice-shop. Acesso em: 27 out. 2026.

PODMAN. Getting Started with Podman. Disponível em: podman.io/docs. Acesso em: 27 out. 2026.

PROMETHEUS AUTHORS. Overview. Disponível em: prometheus.io/docs/introduction/overview/. Acesso em: 27 out. 2026.

PROMETHEUS AUTHORS. First steps with Prometheus. Disponível em: prometheus.io/docs/introduction/first_steps/. Acesso em: 27 out. 2026.

ZABBIX SIA. What is Zabbix. Disponível em: www.zabbix.com/documentation/current/en/manual/introduction/about. Acesso em: 27 out. 2026.

FACULDADES INTEGRADAS DE BAURU. Aula 07 Infraestrutura de TI Tópicos Especiais II. Material didático disponibilizado para a disciplina, 2026.